

Roteiro de Projeto Final

Pipeline de Machine Learning End-to-End

Supabase • DuckDB • DagsHub (DVC) • MLflow • Docker • Render • Streamlit

1. Visão Geral do Projeto

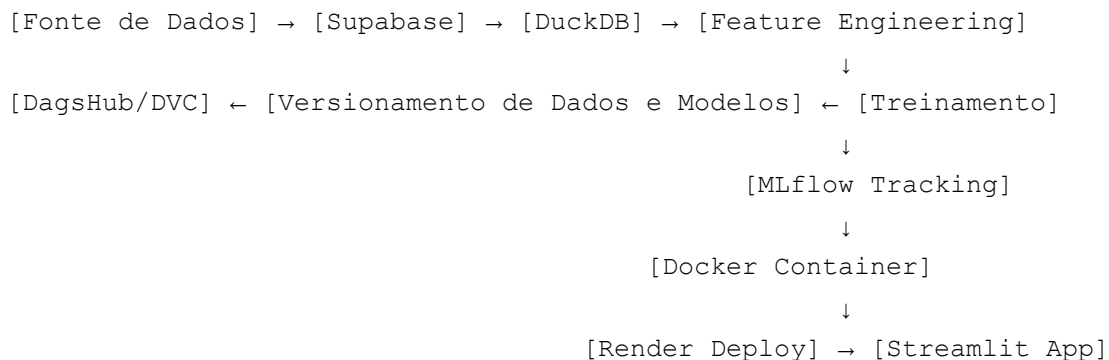
O objetivo deste projeto final é construir um pipeline completo de Machine Learning, indo da ingestão de dados até o deploy de uma aplicação interativa, passando por versionamento, experimentação e containerização. A proposta integra ferramentas modernas de MLOps, permitindo que o aluno exercite na prática um fluxo próximo ao encontrado em ambientes profissionais.

Tema: o domínio do problema é **livre** — fica a critério do aluno ou equipe. O importante é que o problema escolhido envolva dados tabulares (ou estruturáveis) e permita a aplicação de um modelo de machine learning supervisionado ou não supervisionado. A escolha deve considerar disponibilidade de dados, relevância do problema e viabilidade técnica dentro do prazo.

A escolha do tema é também uma oportunidade de demonstrar criatividade e capacidade de formular um problema real a partir de dados disponíveis (datasets públicos, APIs, portais de dados abertos, dados sintéticos, entre outros).

2. Arquitetura Geral

O diagrama abaixo ilustra o fluxo de dados e o papel de cada ferramenta no pipeline:



3. Etapas Detalhadas do Roteiro

Etapa 1 — Definição do Problema e Configuração do Ambiente

Antes de qualquer implementação, defina claramente o problema a ser resolvido, a variável-alvo (quando aplicável), as métricas de avaliação adequadas e a fonte de dados que será utilizada. Documente essas decisões no README do repositório.

Em seguida, crie um repositório no GitHub e conecte-o ao DagsHub, que espelha o repositório e adiciona funcionalidades específicas de MLOps. Configure o ambiente Python com um arquivo requirements.txt contendo as bibliotecas necessárias, como supabase-py, duckdb, dvc, mlflow, scikit-learn, pandas, streamlit e python-dotenv.

Estrutura sugerida de diretórios:

```
projeto-final/
├── data/                # Dados versionados via DVC
├── notebooks/          # EDA exploratória
├── src/
│   ├── ingestion.py    # Ingestão Supabase → DuckDB
│   ├── preprocessing.py # Feature engineering
│   ├── train.py        # Treinamento + MLflow
│   └── predict.py      # Função de inferência
├── app/
│   └── streamlit_app.py # Interface do usuário
├── Dockerfile
├── requirements.txt
├── dvc.yaml             # Pipeline DVC
└── README.md
```

Etapa 2 — Camada de Dados com Supabase

O Supabase funcionará como a fonte de dados transacional do projeto, oferecendo um PostgreSQL gerenciado de forma simples. Crie um projeto no Supabase, modele uma ou mais tabelas conforme o problema escolhido e faça upload do dataset inicial via interface web ou via API.

Atenção: as credenciais (SUPABASE_URL e SUPABASE_KEY) devem ser armazenadas em um arquivo .env que **nunca** deve ser versionado no Git. Inclua-o no .gitignore desde o início.

No script ingestion.py, conecte-se ao Supabase usando a biblioteca oficial e faça o download dos dados para um DataFrame, servindo como ponto de partida do pipeline.

Etapa 3 — Processamento Analítico com DuckDB

O DuckDB entra como engine analítica embutida, ideal para transformações SQL rápidas em memória sem precisar de um servidor dedicado. Ele se integra nativamente com pandas e arquivos Parquet, o que o torna excelente para a camada de processamento.

Utilize o DuckDB para realizar agregações, joins, limpeza e feature engineering via SQL, diretamente sobre os DataFrames extraídos do Supabase. O fluxo típico consiste em carregar o DataFrame no DuckDB, executar as queries de transformação e salvar o resultado como arquivo Parquet em data/processed/ para versionamento posterior.

Etapa 4 — Versionamento com DagsHub e DVC

Inicialize o DVC no projeto com o comando `dvc init` e configure o remote apontando para o armazenamento do DagsHub. Em seguida, adicione os arquivos de dados ao rastreamento do DVC utilizando `dvc add`, e crie um arquivo `dvc.yaml` descrevendo o pipeline em estágios (ingestão, processamento e treinamento).

Essa configuração garante reprodutibilidade: qualquer pessoa pode clonar o repositório, rodar `dvc repro` e reconstruir todo o pipeline do zero, com dados e artefatos consistentes.

Etapa 5 — Experimentação com MLflow

Configure o MLflow para usar o tracking server do DagsHub, que oferece MLflow integrado gratuitamente. No script `train.py`, envolva o treinamento em um bloco `mlflow.start_run()` e

registre parâmetros (hiperparâmetros do modelo), métricas (apropriadas ao problema — RMSE/MAE/R² para regressão, accuracy/F1/AUC para classificação, etc.), artefatos (o próprio modelo serializado) e tags descritivas.

Treine pelo menos três modelos diferentes para permitir comparação justa na UI do MLflow hospedada no DagsHub. Registre o melhor modelo no Model Registry para facilitar o carregamento posterior pela aplicação.

Etapa 6 — Aplicação Streamlit

Crie uma interface amigável em `streamlit_app.py` com os seguintes elementos mínimos: campos de entrada para as features do modelo; um botão de previsão que carrega o modelo registrado no MLflow e exibe o resultado; e, opcionalmente, uma aba com visualizações exploratórias dos dados (histogramas, mapas, estatísticas descritivas — o que fizer sentido para o domínio escolhido).

Considere também mostrar métricas do modelo em produção, informações sobre as features esperadas e exemplos de uso, tornando a aplicação autoexplicativa.

Etapa 7 — Containerização com Docker

Escreva um Dockerfile partindo de uma imagem base como `python:3.11-slim`. O arquivo deve instalar as dependências listadas no `requirements.txt`, copiar o código da aplicação para dentro do container e expor a porta 8501, padrão do Streamlit. O comando final deve iniciar a aplicação com:

```
streamlit run app/streamlit_app.py --server.port=$PORT --
server.address=0.0.0.0
```

Teste localmente com `docker build` e `docker run` antes de fazer deploy — é muito mais fácil debugar problemas no ambiente local do que em produção.

Etapa 8 — Deploy no Render

No Render, crie um novo Web Service conectado ao seu repositório do GitHub, escolha a opção de deploy via Docker e configure as variáveis de ambiente necessárias (credenciais do Supabase, token do DagsHub para baixar o modelo do MLflow, etc.) na seção Environment do serviço.

O Render fará o build automático a cada push na branch principal, habilitando um fluxo básico de CI/CD. Ao final, a aplicação estará disponível publicamente em uma URL fornecida pela plataforma.

4. Entregáveis Finais

Ao final do projeto, espera-se que sejam entregues:

- Repositório no GitHub/DagsHub contendo o código-fonte e o histórico de commits.
- Dados versionados via DVC, armazenados no remote do DagsHub.
- Experimentos rastreados no MLflow, com comparação de ao menos três modelos.
- Imagem Docker funcional, testada localmente antes do deploy.
- Aplicação Streamlit acessível publicamente via Render.
- README completo, explicando o problema escolhido, a arquitetura adotada, como reproduzir o pipeline e as principais decisões técnicas tomadas.

5. Critérios de Avaliação Sugeridos

Para orientar o desenvolvimento, considere que um bom projeto demonstra:

- Justificativa clara do problema escolhido e da abordagem adotada.
- Qualidade do código: organização, modularização e documentação.
- Uso adequado de cada ferramenta, respeitando o seu propósito (sem forçar integrações desnecessárias).
- Reprodutibilidade do pipeline por terceiros.
- Comparação honesta entre modelos, com métricas apropriadas ao problema.
- Aplicação final funcional, usável e estável.
- Documentação suficiente para que outra pessoa consiga rodar o projeto do zero.

6. Dicas Importantes

Segurança: nunca comite credenciais no repositório. Utilize arquivos `.env` e o `.gitignore` de forma rigorosa.

Iteração: comece simples. Coloque um pipeline básico funcionando de ponta a ponta antes de otimizar qualquer etapa individualmente.

Documentação: escreva o README conforme o projeto avança. É muito mais fácil documentar decisões durante o desenvolvimento do que reconstruí-las no final.

Testes incrementais: valide cada etapa isoladamente antes de integrar ao pipeline completo.

Planos gratuitos: atente-se aos limites dos planos gratuitos das ferramentas utilizadas (Supabase, Render, DagsHub) e dimensione o projeto de acordo.